

28 — BACKPROPAGATION NEURAL NETWORK USING SCIKIT-LEARN

Aim

To implement a multilayer perceptron neural network using scikit-learn's MLPClassifier and classify the Iris dataset using backpropagation.

Algorithm + Procedure

1. Load the Iris dataset using scikit-learn.
2. Separate the input features and target classes.
3. Divide the dataset into training and testing sets.
4. Standardize the input features using StandardScaler.
5. Create an MLPClassifier with one hidden layer containing 10 neurons.
6. Train the neural network using the training dataset.
7. The network uses backpropagation to update its weights.
8. Predict the classes of the test dataset.
9. Compare the predicted classes with the actual classes.
10. Calculate and display the test accuracy.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
```

```

)
# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Create MLP classifier
mlp = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=1
)
# Train model
mlp.fit(X_train, y_train)
# Predict test data
y_pred = mlp.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

```

Output

Accuracy: 0.9555555555555556

Result

The multilayer perceptron neural network was successfully implemented using scikit-learn. The model achieved approximately **95.5% test accuracy** on the Iris dataset.